



Platform Engineering Day **EUROPE**

AMSTERDAM, THE NETHERLANDS

23 MARCH 2026

#PLATENGDAY

How (not) To Stage Your (Kubernetes) Platform

Promotion failed

Your speaker



Green tone
may vary

I „do cloud/platform stuff“

Nicolai Ort

Platform Engineer
ODIT.Services

@nicolaiort

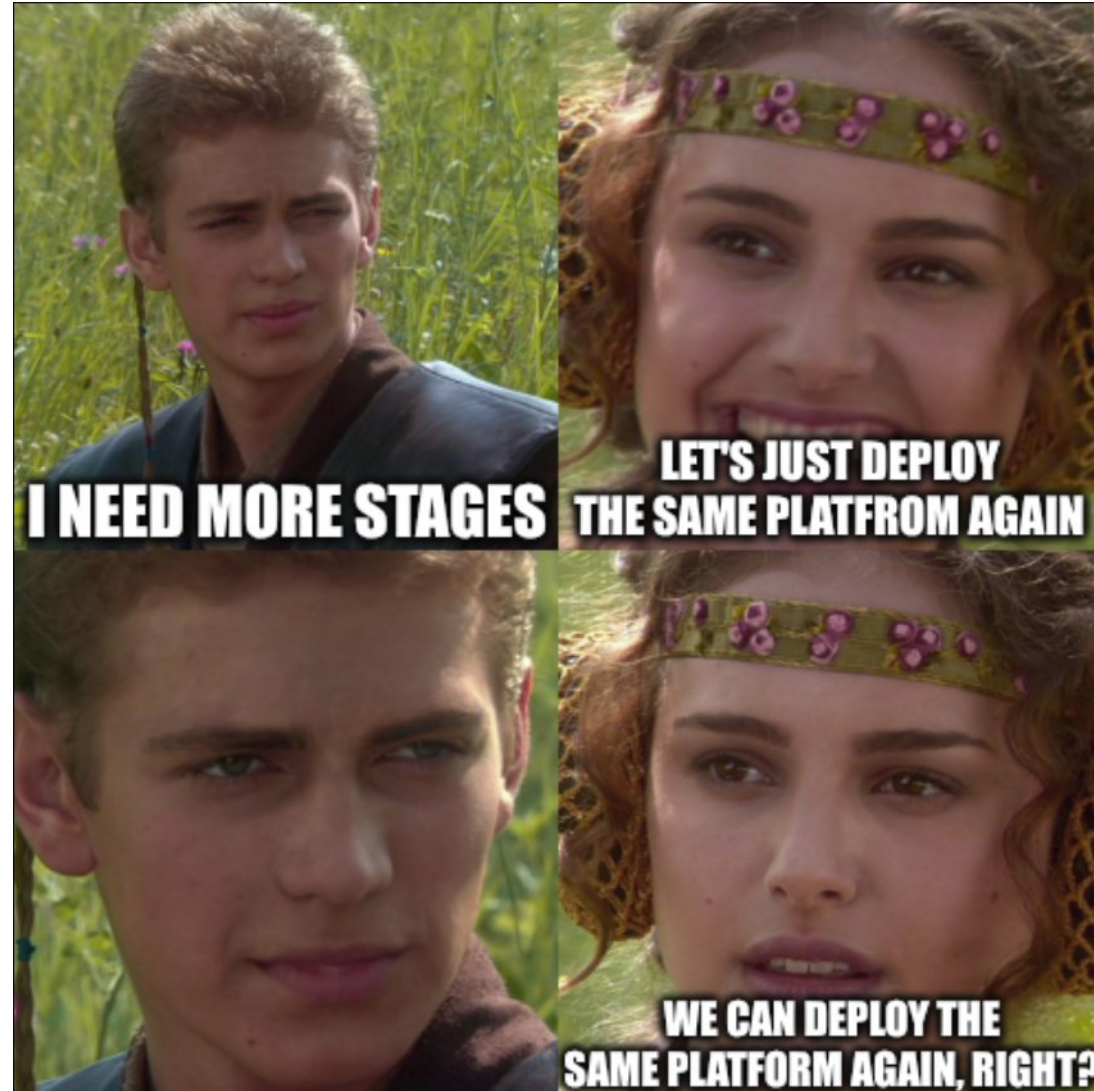
Staging

It's simple



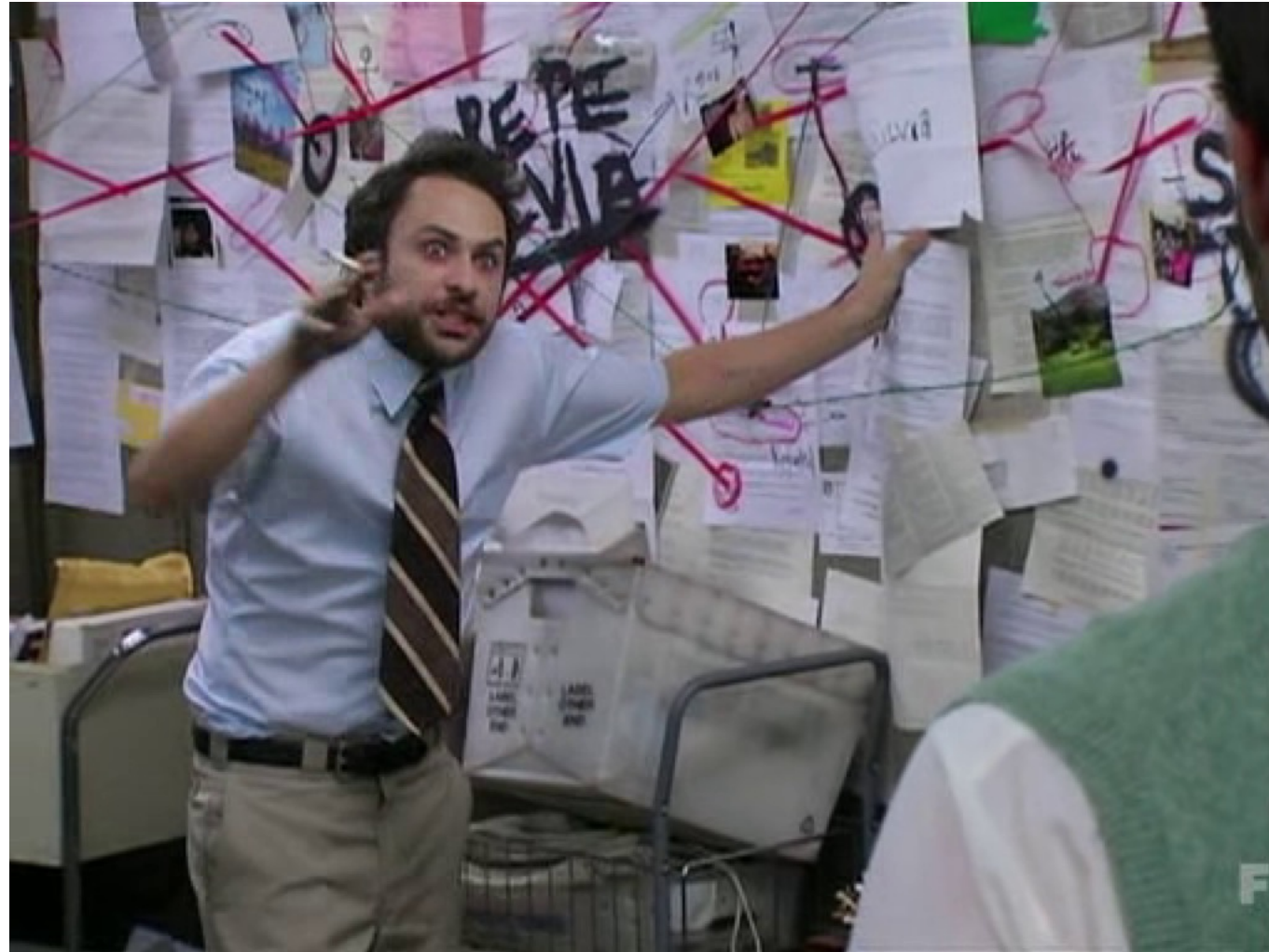
It's simple

isn't it?



What is a platform

What is a platform



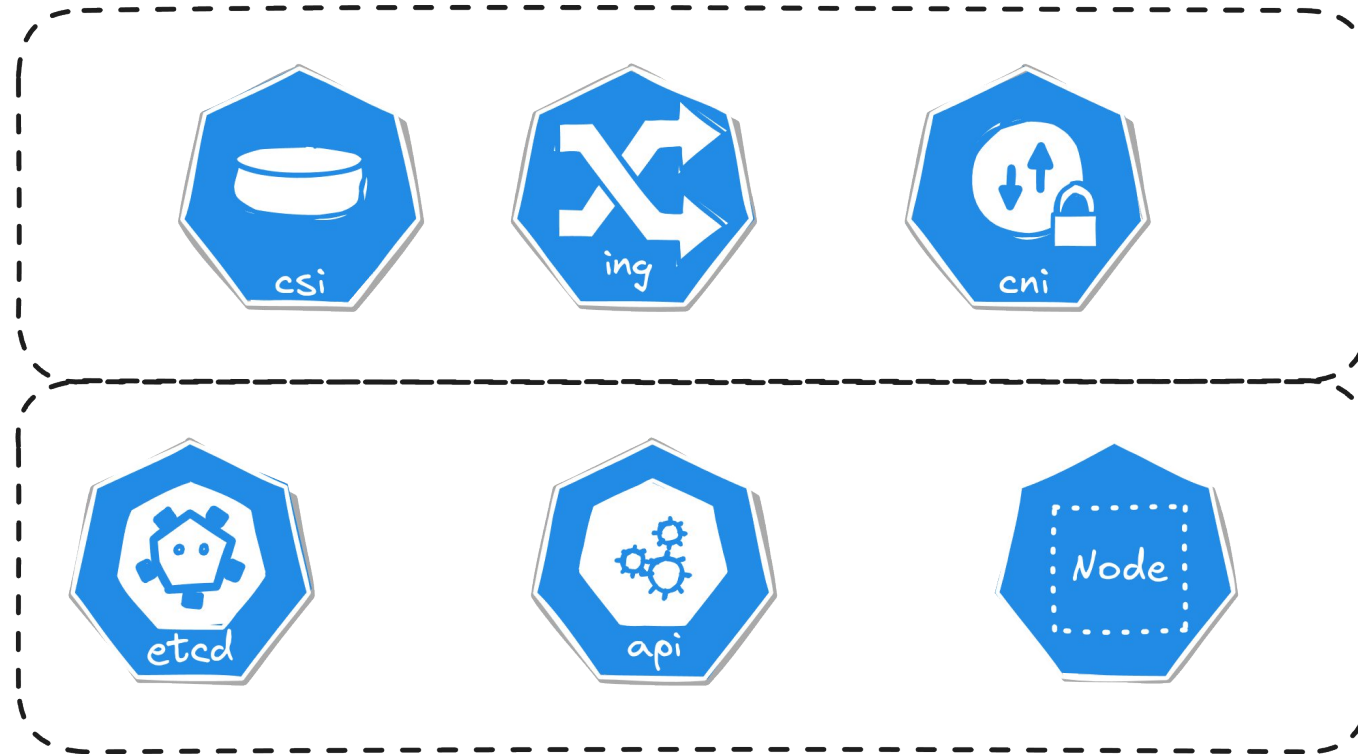
What is our platform?



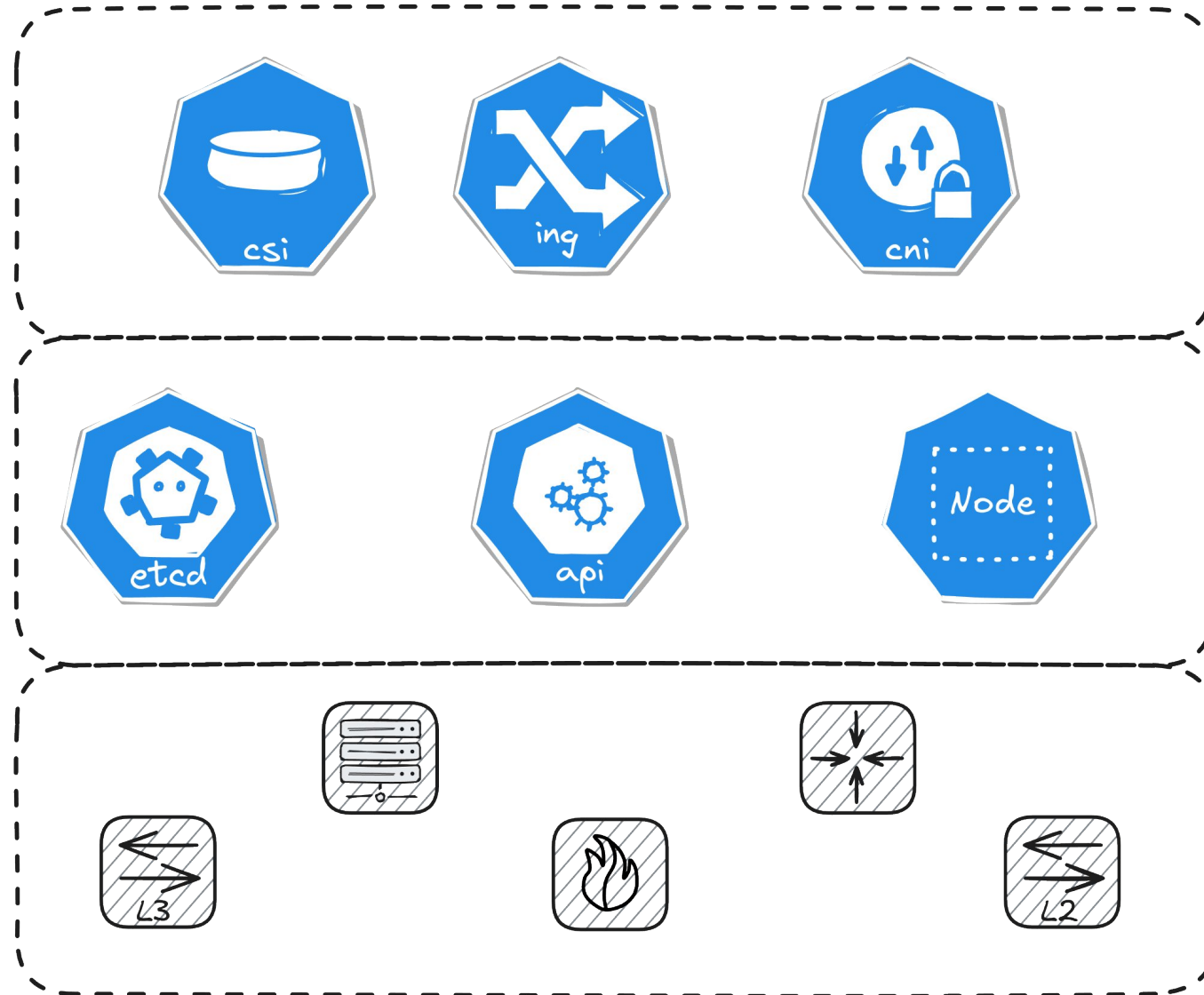
What is our platform made of



What is our platform made of



What is our platform depending on



And what about these „stages“

The mysterious stage



The mysterious stage

A copy of our stack

And how many do i need

And how many do i need

1. Developers: We need DEV to test and PROD to run our stuff

And how many do i need

1. Developers: We need DEV to test and PROD to run our stuff
2. QA: We need QA

And how many do i need

1. Developers: We need DEV to test and PROD to run our stuff
2. QA: We need QA
3. Platform: It would be nice to evaluate Changes

And how many do i need

1. Developers: We need DEV to test and PROD to run our stuff
2. QA: We need QA
3. Platform: It would be nice to evaluate changes
4. Infra: Well we need to test new firmware updates

And how many do i need

1. Developers: We need DEV to test and PROD to run our stuff
 2. QA: We need QA
 3. Platform: It would be nice to evaluate changes
 4. Infra: Well we need to test new firmware updates
- -
 -

And how many do I need

It depends!

(only carry what you can)

And how many do I need:

My recommendation

1. Fullfill you customers needs
2. Add a stage for yourself
3. Try to combine stages if possible
4. Keep things reasonable







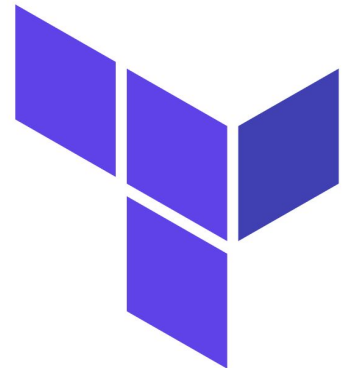
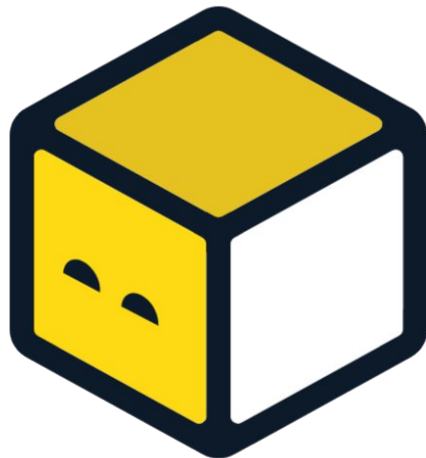
State management

Part 1: Apply state

Tooling



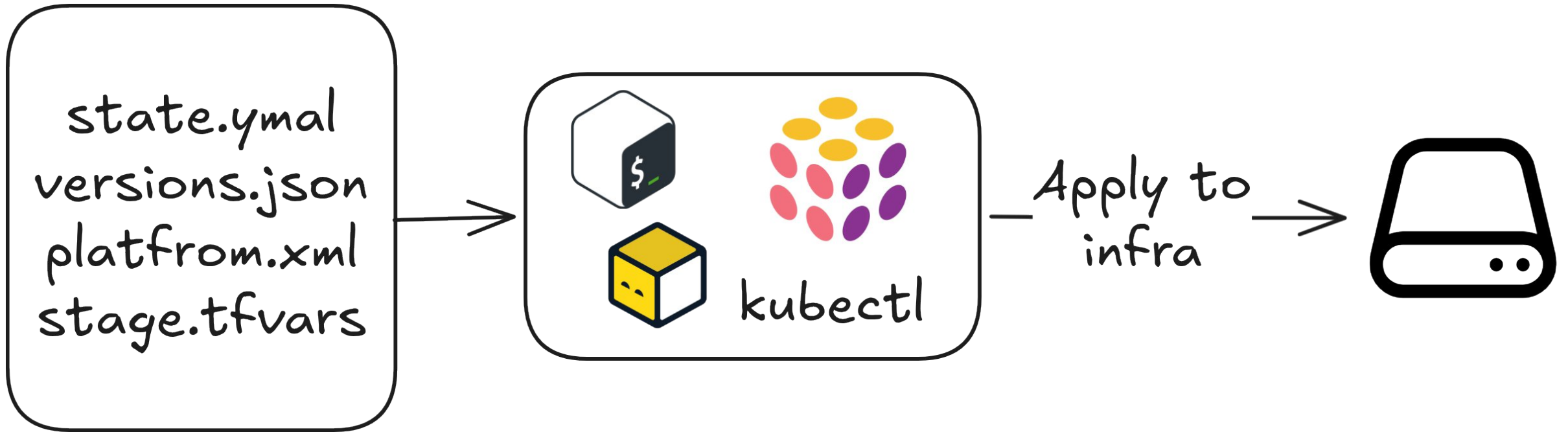
It does not really matter
you always need a way to apply changes



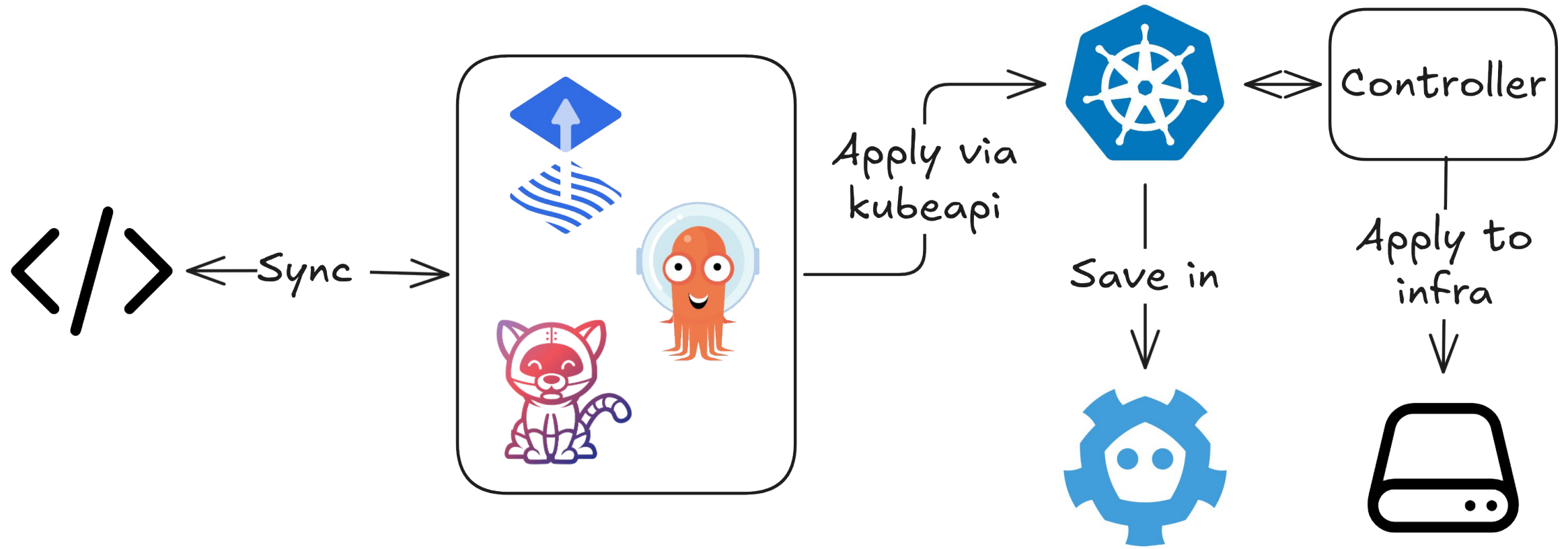
A (custom) statefile + magic



A (custom) statefile + magic



Your favorite GitOps tool + Operators



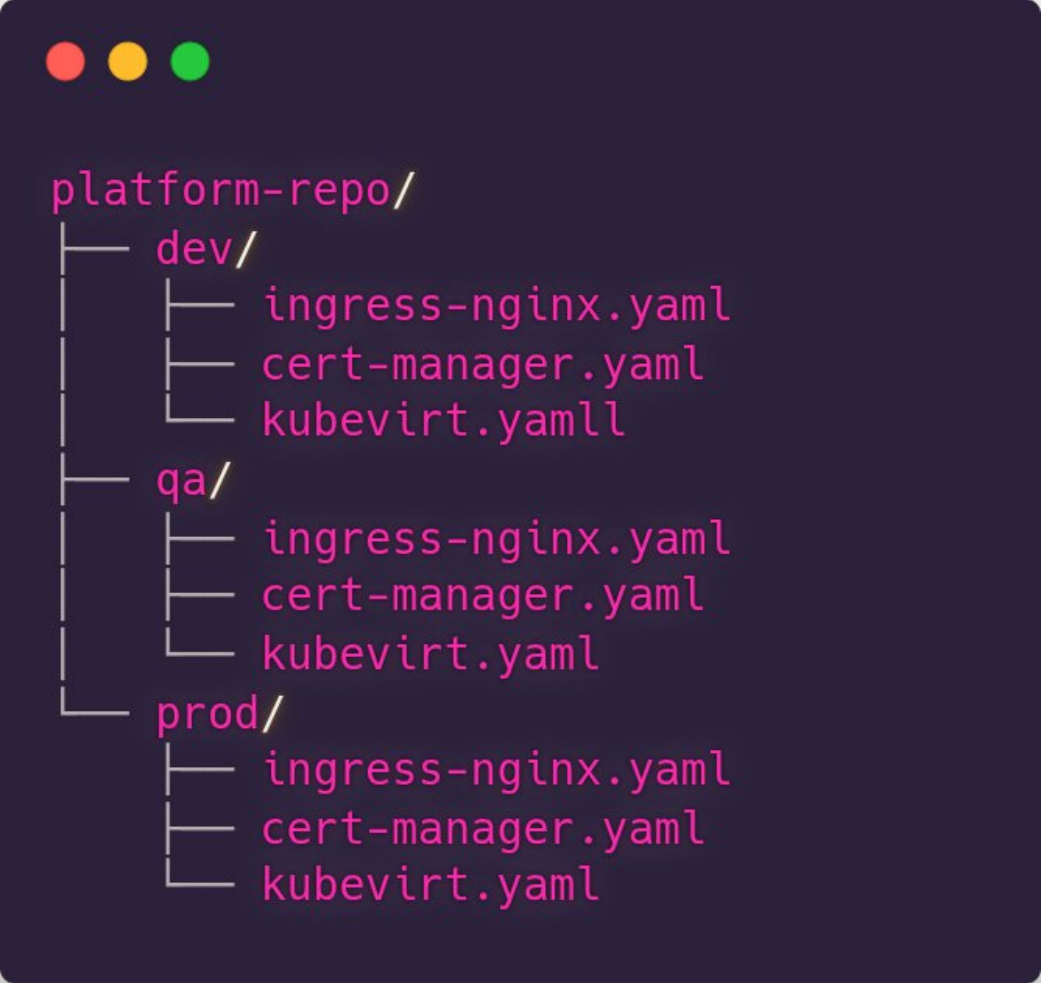
At the end of the day



State management

Part 2: Organize state

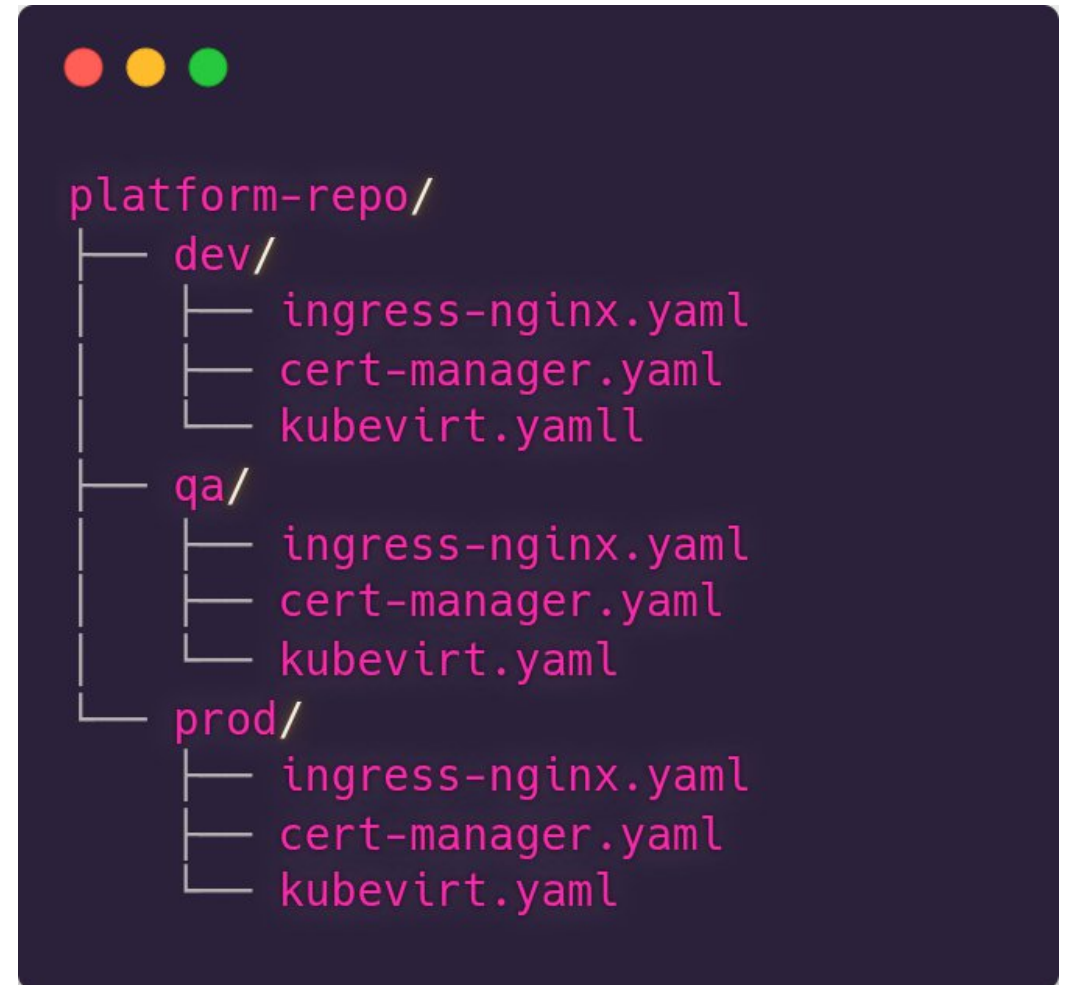
The duplication-approach



```
platform-repo/  
├── dev/  
│   ├── ingress-nginx.yaml  
│   ├── cert-manager.yaml  
│   └── kubevirt.yaml  
├── qa/  
│   ├── ingress-nginx.yaml  
│   ├── cert-manager.yaml  
│   └── kubevirt.yaml  
└── prod/  
    ├── ingress-nginx.yaml  
    ├── cert-manager.yaml  
    └── kubevirt.yaml
```

The duplication-approach

- + Small teams
- + Early start
- + Per-stage flexibility
- Duplicate files
- High drift risk
- Hard to scale



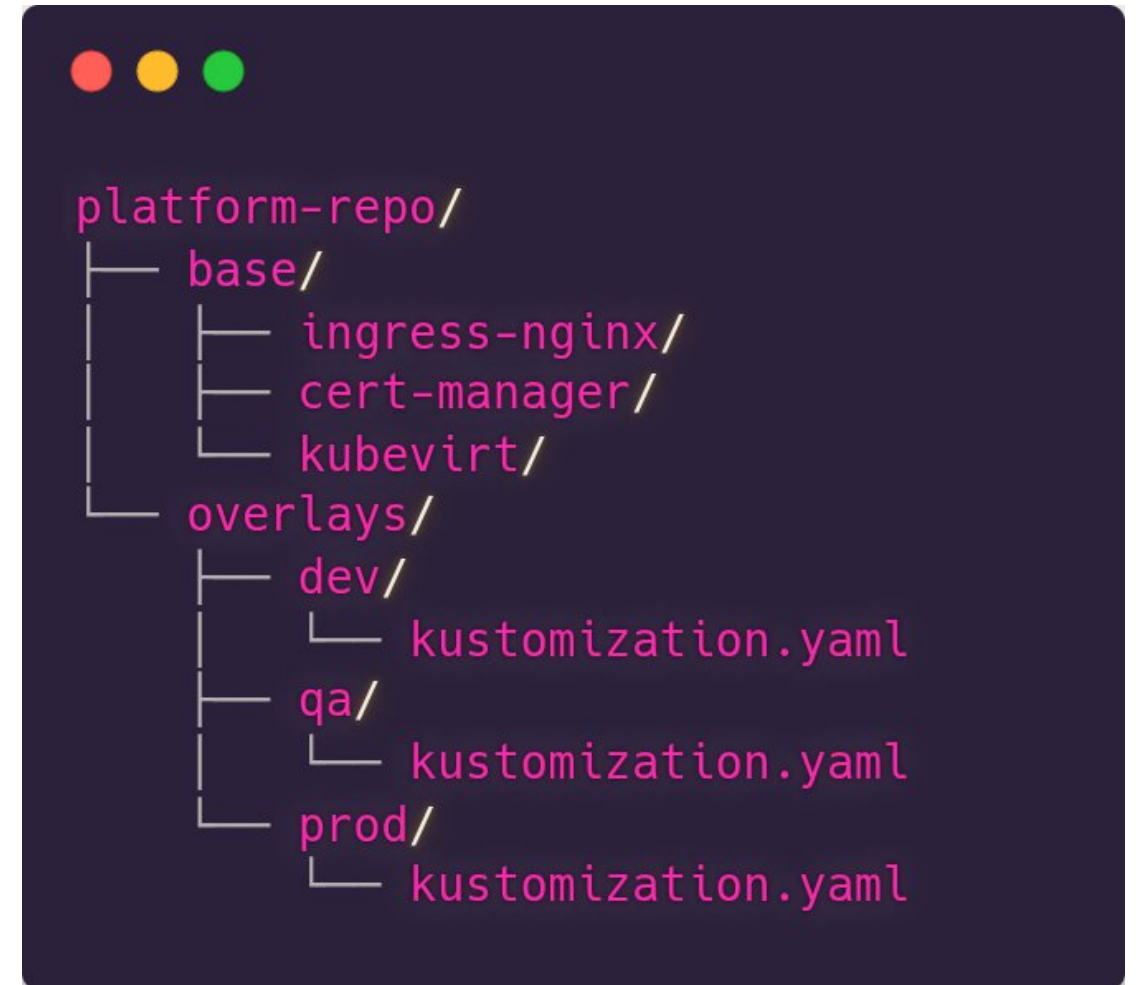
Extracting common files



```
platform-repo/  
├── base/  
│   ├── ingress-nginx/  
│   ├── cert-manager/  
│   └── kubevirt/  
└── overlays/  
    ├── dev/  
    │   └── kustomization.yaml  
    ├── qa/  
    │   └── kustomization.yaml  
    └── prod/  
        └── kustomization.yaml
```

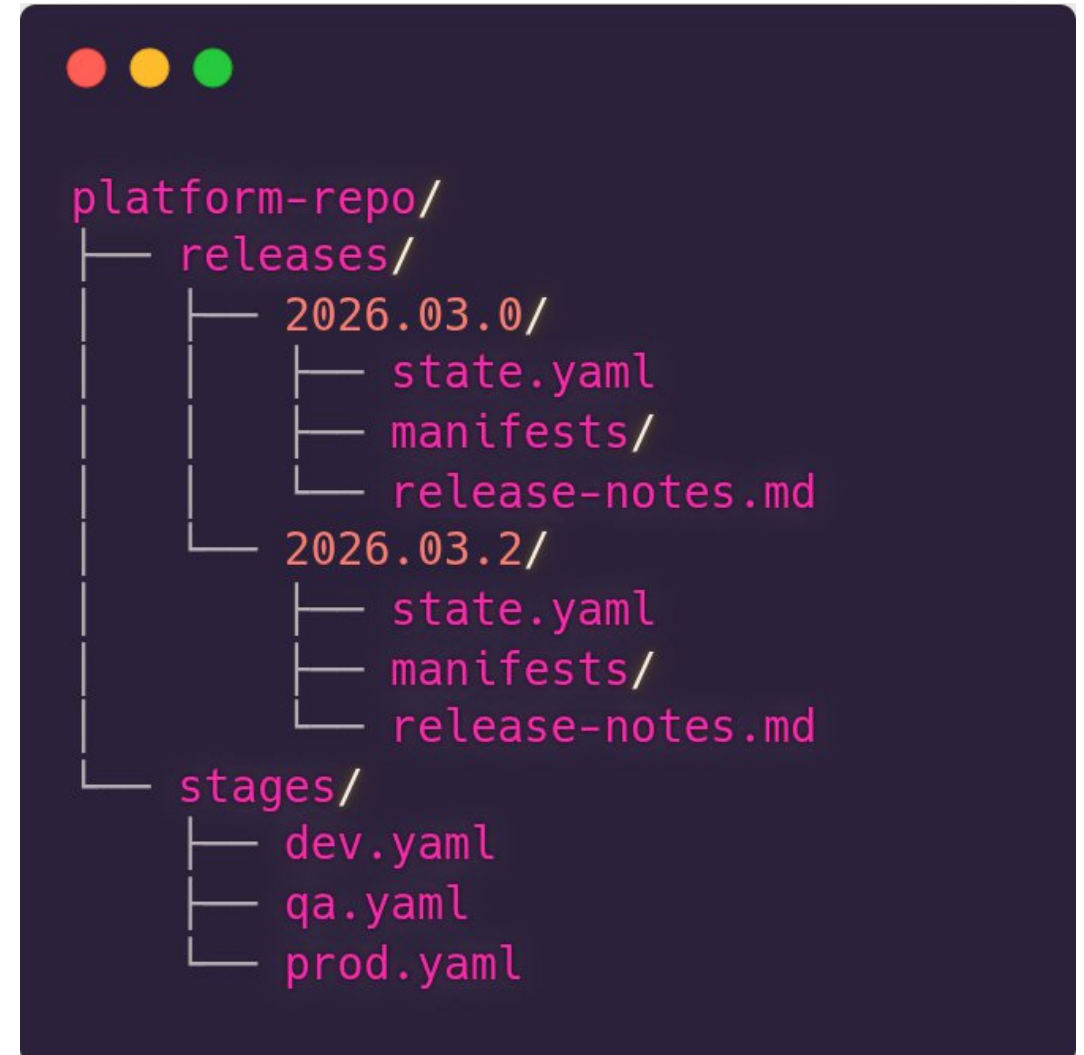
Extracting common files

- + Less duplication
- + Separation of share/specific
- Can become messy
- Only for similar stages



Git Tags

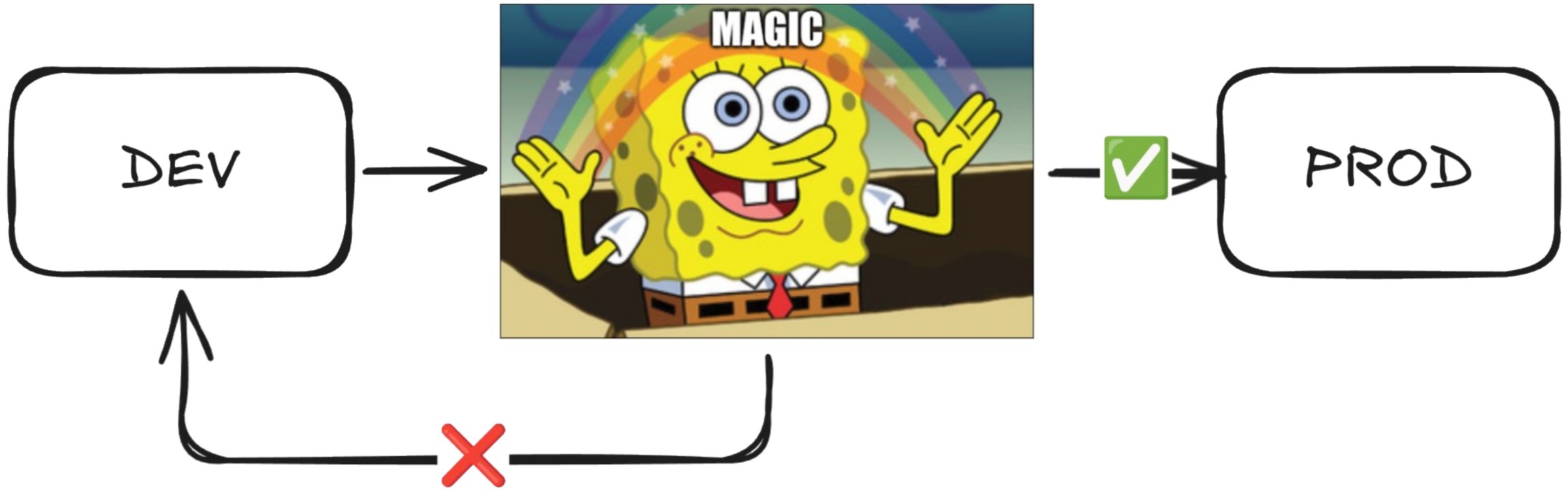
- + Clear release artifact
- + Easy to audit/diff
- + Simplifies rollback
- Less flexible
- Additional engineering overhead



How do we promote state

Aka get changes from DEV to QA to PROD?

Your dreams



Manual promotion



```
cp dev/state.yaml qs/state.yaml  
git commit -m "chore: Promoted v1"
```

Manual promotion

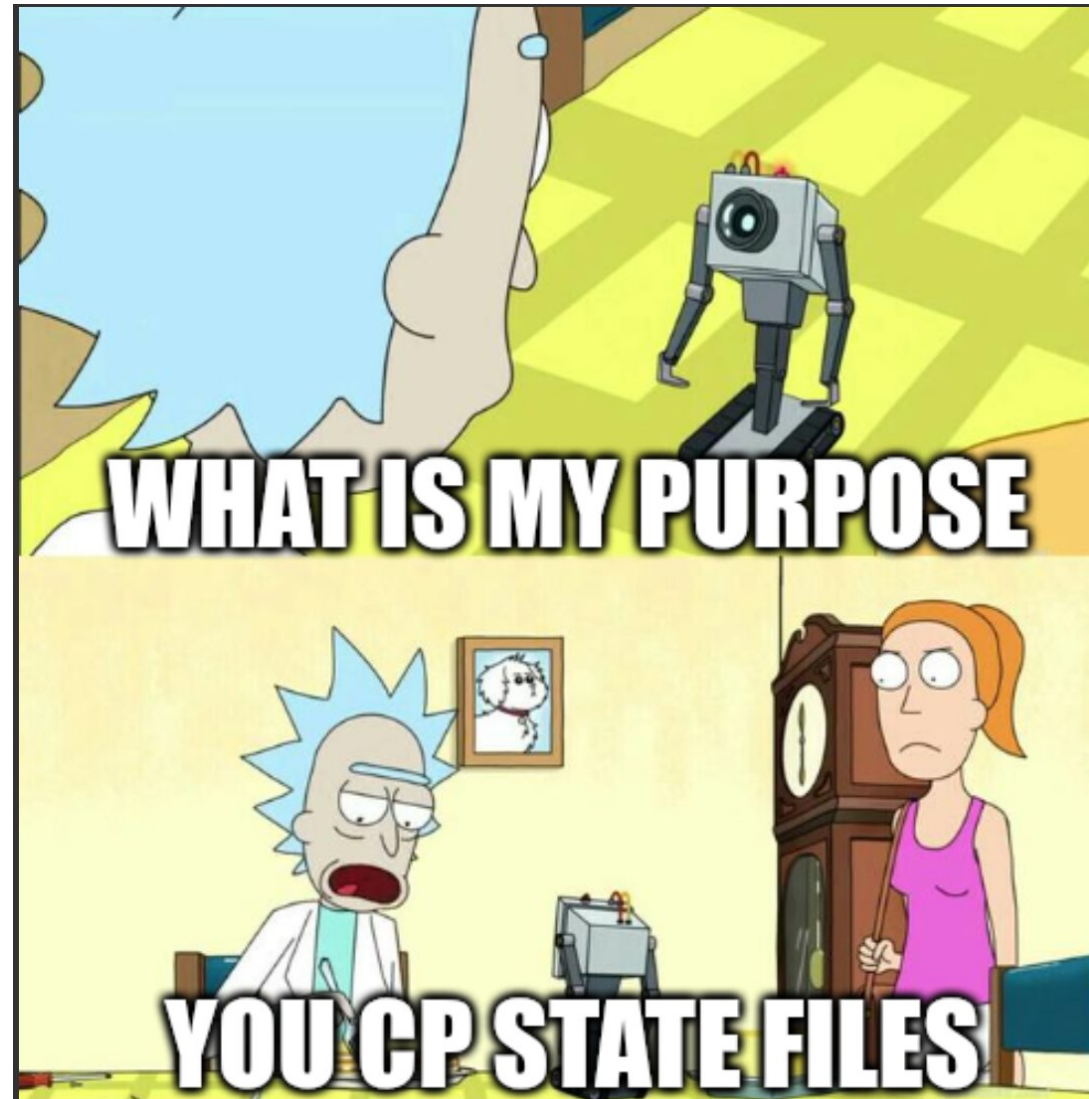
- + Small teams
- + Early days

- Compliance
- Scale
- Consistency



```
cp dev/state.yaml qs/state.yaml  
git commit -m "chore: Promoted v1"
```

Bare minimum: automatic promotion



Static Delays



Static delays



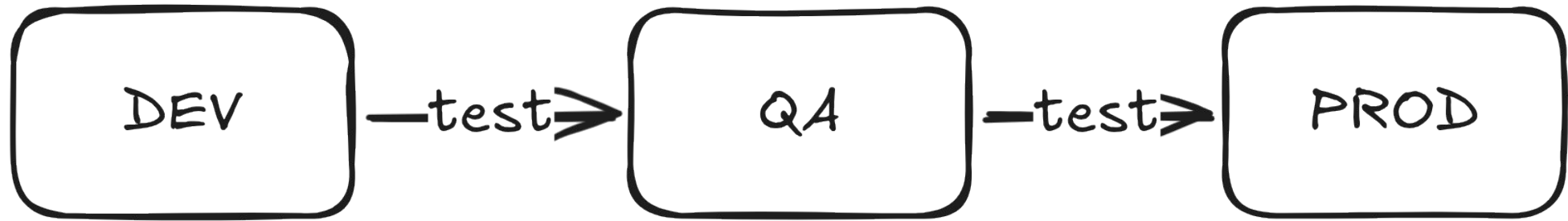
Static delays

- + Low complexity
- + Basic safety
- + Good first step

- Emergencies
- Issue driven validation



E2E-Gates



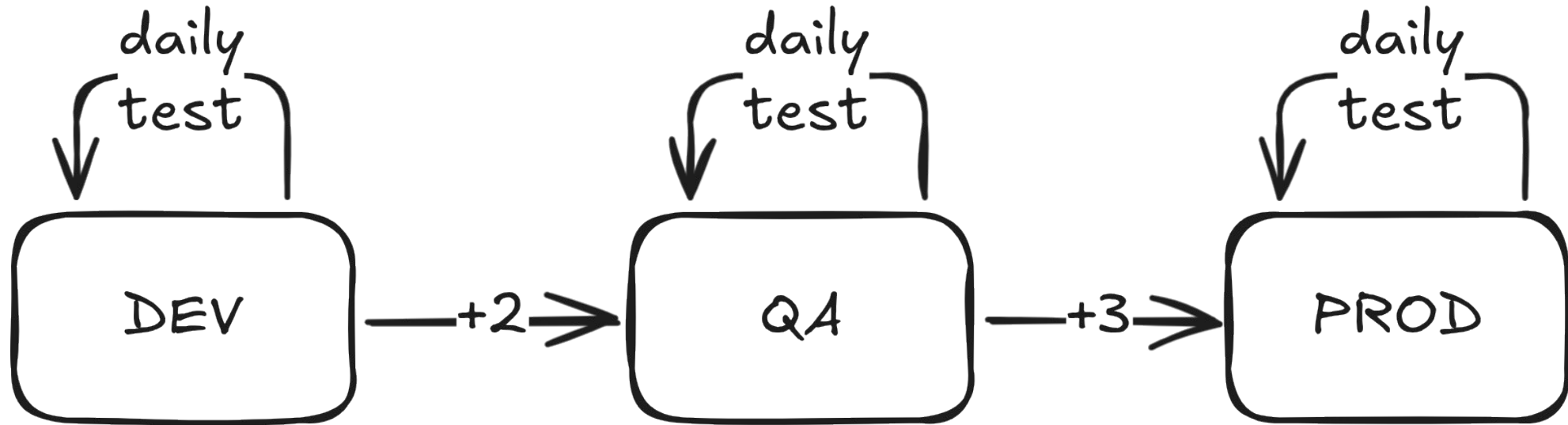
E2E-Gates

- + High confidence
- + Repeatable

- Complex to implement
- Needs maintainance
- 100%-CodeCov-Problem



A healthy combination



Coping with reality

Something always breaks



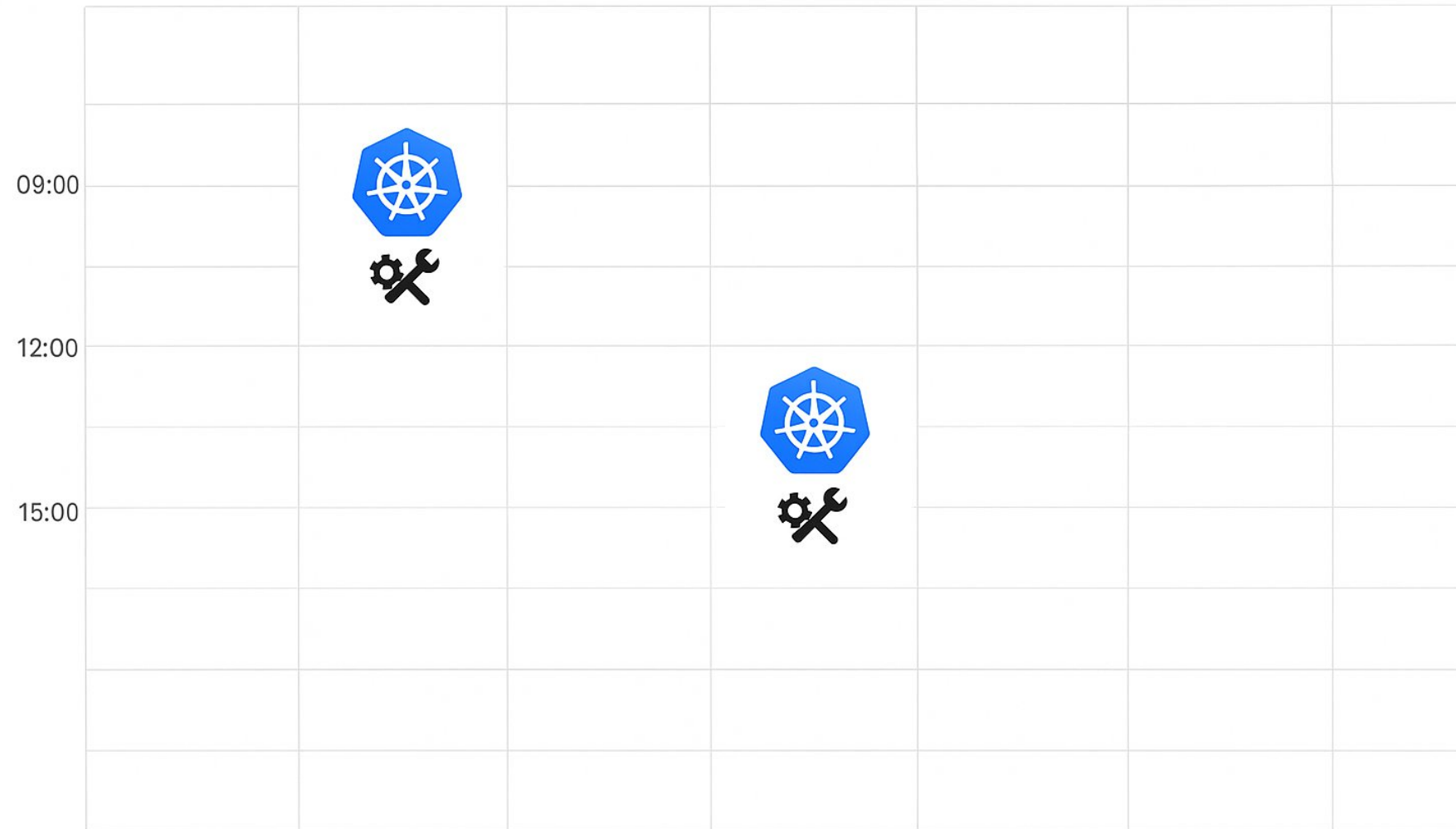
Fast-Tracking

Backup/Restore

Additional

A quick sidequest in stakeholder communications

A likely baseline



Minor improvements

Blog



Placeholder text for the first blog entry.



Placeholder text for the second blog entry.

A recommendation

Overview

Stage	Supported Kubernetes Versions			OS Version	Delay	Last update
Infra	1.33	1.34	1.35	flatcar:4459.2.4	–	2026-03-20
LAB	1.33	1.34	1.35	flatcar:4459.2.4	1 day	2026-03-21
DEV	1.33	1.34	1.35	flatcar:4459.2.4	2 days	2026-03-23
QA	1.32	1.33	1.34	flatcar:4459.2.3	3 days	2026-02-02
PROD	1.32	1.33	1.34	flatcar:4459.2.3	4 days	2026-03-06

TL;DL

Stop yapping already

TL;DL

1. Find out what actually is a part of your platform

TL;DL

1. Find out what actually is a part of your platform
2. Include all major dependencies in your staging

TL;DL

1. Find out what actually is a part of your platform
2. Include all major dependencies in your staging
3. Choose some kind of versioned state

TL;DL

1. Find out what actually is a part of your platform
2. Include all major dependencies in your staging
3. Choose some kind of versioned state
4. Start with delays and add automatic tests

TL;DL

1. Find out what actually is a part of your platform
2. Include all major dependencies in your staging
3. Choose some kind of versioned state
4. Start with delays and add automatic tests
5. Tell your users what is happening when to increase trust

Thank you!

Have a great
lunch break
Platform Engineering Day
KubeCon/CloudNativeCon



Slides & More: <https://github.com/nicolaiort/pedeu2026-promotion-failed>



Platform Engineering Day **EUROPE**

AMSTERDAM, THE NETHERLANDS

23 MARCH 2026

#PLATENGDAY